

Programming Basics 1



06: Klassen und Objekte

Sommersemester 2025

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

simon.dewert@haw-hamburg.de

Übersicht dieser Lehreinheit

1. Klassendeklaration
2. Instanziierung
3. Attribute
4. Methoden
5. Überladung
6. Rückgabewerte
7. Statische Elemente
8. Zugriffsmodifizierer

Prozedural

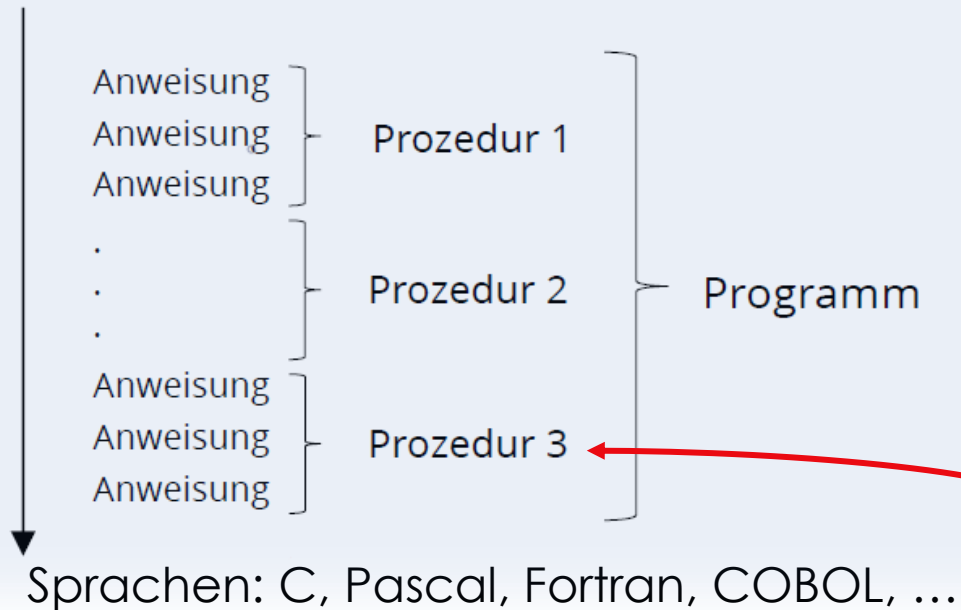
vs.

Objektorientiert

Ausgangspunkt:

Funktionsweise des Rechners

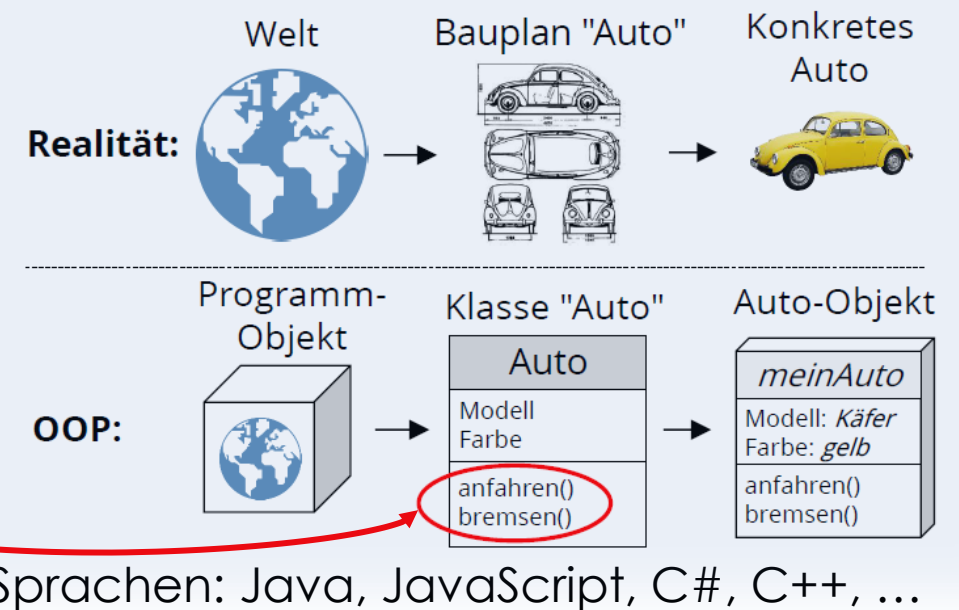
Programm = Endliche Folge von Anweisungen



Ausgangspunkt:

Funktionsweise der realen Welt

Programm = Nachbildung der realen Welt mit Objekten



Klassendeklaration

Die Deklaration einer Klasse bewirkt die Spezifikation eines neuen, **komplexen Datentyps**. Durch diesen werden sog. Attribute (Datenelemente) und Methoden gekapselt:

Schlüsselwort zur
Definition einer Klasse

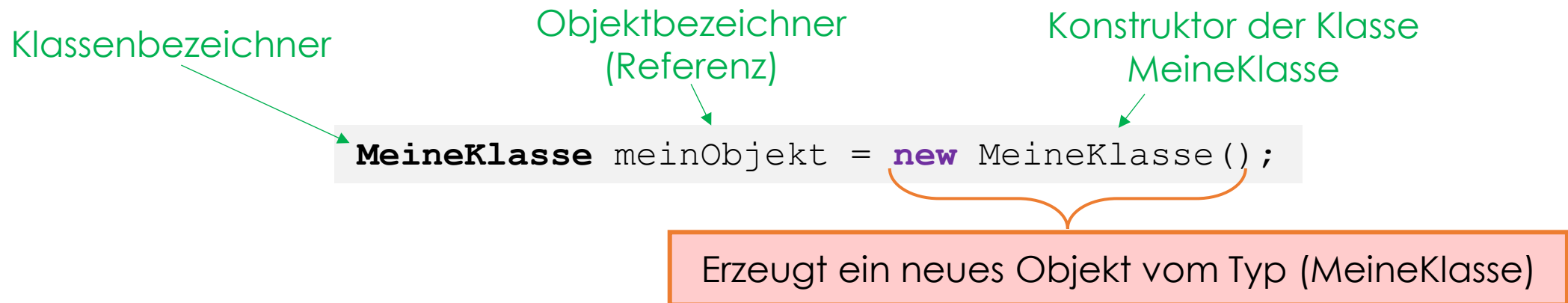
Klassenbezeichner
(GROSSER Anfangsbuchstabe)

```
class MeineKlasse
{
    [Attribute der Klasse]
    [Konstruktoren der Klasse]
    [Methoden der Klasse]
}
```

Instanziierung (erzeugt ein Objekt)

Eine Klasse kann als eine Art "Bauplan" verstanden werden, mit dem beliebig viele **Objekte** (auch "Instanzen" genannt) also Realisierungen dieses Bauplans erzeugt werden können.

Generell wird zunächst eine **Variable** des Datentyps unter Angabe eines Bezeichners angelegt. Dieser Variable wird dann eine Instanz der Klasse durch die Angabe des Schlüsselworts **new**, gefolgt vom Aufruf eines sog. **Konstruktors*** der Klasse, zugewiesen:



Attribute (Klassenvariablen)

Die Attribute (auch "Membervariablen" oder "Felder" genannt) sind die Variablen bzw. **Datenelemente** einer Klasse.

Jede Instanz der Klasse enthält einen eigenen Satz an Kopien der in der Klasse definierten Attribute. Es kann sich dabei um sowohl um **elementare** als auch um **komplexe** Datentypen handeln.

Beispiel:

```
class MeineKlasse
{
    int eineGanzzahl = 0;
    double eineDezimalzahl = 2.0;
    String eineZeichenkette = "abc";
    KlasseXYZ einObjektXYZ = null;
}
```

Explizite Initialisierung; sonst Vergabe des Default-Werts des jeweiligen Datentyps

Attribute der Klasse MeineKlasse

Aufgabe 6.1



Legen Sie zunächst ein neues Java-Projekt namens **vorlesung_06** an!
Schreiben Sie dann ein Programm/Klasse **Pferd**.
Die Klasse Pferd soll folgende Attribute erhalten:

Pferd
name: <i>String</i> geschwindigkeit: <i>double</i> strecke: <i>double</i> hunger: <i>boolean</i>

Erzeugen Sie innerhalb Ihrer main-Methode ein Objekt dieser Klasse und weisen Sie diesem mit Hilfe des Scanner Werte zu.

(optional) legen Sie eine zusätzliche Klasse an, in welche die main()-Methode ausgelagert wird.

Konstruktor

Konstruktor sind spezielle Funktionen (sog. "Methoden"), die **immer den Namen der Klasse** tragen und der Instanziierung dienen.

Sie werden darüber hinaus genutzt, um Attributen bestimmte **Anfangswerte** zuzuweisen:

```
class MeineKlasse
{
    int eineZahl = 0;
    MeineKlasse (int eineZahl)
    {
        this.eineZahl = eineZahl;
    }
}
```

Explizite Initialisierung (noch **VOR** der Instanziierung)

Übergabe-Parameter (Argument)

Konstruktor der Klasse MeineKlasse mit
(erneuter) Wertzuweisung

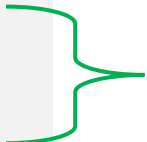
```
MeineKlasse meinObjekt = new MeineKlasse(10);
```


Default-Konstruktor

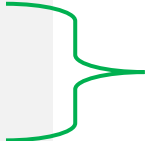
Sollte **kein Konstruktor** definiert worden sein, so verfügt jede Klasse jedoch zumindest über den sogenannten **Default-Konstruktor** (Konstruktor OHNE Übergabeparameter).

Wurden Konstruktoren definiert, muss der **Default-Konstruktor** – falls gewünscht – ebenfalls **ausdrücklich** definiert werden:

```
class MeineKlasse{  
    int eineZahl = 0;  
  
    MeineKlasse () {  
        eineZahl = 10;  
    }  
  
    MeineKlasse (int zahl) {  
        eineZahl = zahl;  
    }  
}
```



Default-Konstruktor (hier mit fester Zuweisung eines Defaultwerts)



Konstruktor (mit variabler Wertzuweisung)

Aufgabe 6.2



Ergänzen Sie Ihr Programm **Pferd** um einen Konstruktor mit zwei Übergabewerten, welcher direkt die beiden Attribute (name und geschwindigkeit) mit den übergebenen Werten befüllt. Jedes Pferd soll erst mal satt sein und somit das Attribut hunger auf false gesetzt werden.

Beispiel:

```
Pferd jolly = new Pferd("Jolly Jumper", 1.0);
```

jolly.name soll jetzt **Jolly Jumper** sein

jolly.geschwindigkeit soll jetzt **1,0** sein

jolly.strecke soll jetzt **0,0** sein

jolly.hunger soll jetzt **false** sein

Konstruktor vom Konstruktor aufrufen

Mit dem Schlüsselwort `this()` kann innerhalb eines Konstruktors ein anderer Konstruktor aufgerufen werden. In der Regel wird vom „kleinsten“ zum „größten“ Konstruktor übergeben.

Dieses Vorgehen kann dabei helfen doppelten Code zu vermeiden.

```
class MeineKlasse{  
    int eineZahl = 0;  
  
    MeineKlasse () {  
        this(10);  
    }  
  
    MeineKlasse (int zahl) {  
        eineZahl = zahl;  
    }  
}
```

Default-Konstruktor (hier mit fester Zuweisung eines Defaultwerts)
Defaultwert wird an den spezielleren Konstruktor weitergeleitet

Konstruktor (mit variabler Wertzuweisung)

`this()` kann nur an **erster Stelle** im Konstruktor aufgerufen werden!

Ergänzung zu **this**

this() Konstruktor Aufruf.

this (ohne Klammer) steht für das **eigene Objekt**.

Wofür kann **this** verwendet werden?

- Das aktuelle Objekt kann übergeben werden (**move(this);**)
- Bei gleichnamigen Attributen/Variablen kann mit **this.attributsName** explizit das verdeckte Attribut angesprochen werden.

```
class MeineKlasse{  
    int eineZahl ← 0;  
  
    MeineKlasse (int eineZahl){  
        System.out.println(this.eineZahl);  
        System.out.println(eineZahl);  
        this.eineZahl = eineZahl;  
    }  
}
```

Objekte löschen (!?)

Objekte können in Java **nicht direkt gelöscht** werden.

Java überprüft im Hintergrund mit dem **Garbage Collector** kontinuierlich alle Objekte.

Wenn festgestellt wird, dass ein Objekt in **keiner Variable mehr referenziert ist**, wird das Objekt **gelöscht**.

siehe Beispiel

```
public class MeineKlasse {  
    public static void main(String[] args) {  
  
        // erzeuge ein Objekt vom Typ MeineKlasse  
        MeineKlasse obj = new MeineKlasse();  
  
        // mit obj kann jetzt auf das neue Objekt zugegriffen werden.  
  
        // Die Variable obj wird jetzt geleert.  
        // Wir haben jetzt keine Referenz mehr auf das erzeugte Objekt.  
        obj = null;  
    }  
}
```

Methodendeklaration

Methoden implementieren die "Verhaltensweise" der Objekte und können nur **innerhalb einer Klasse** definiert werden.

Rückgabewert (hier: leer, kann aber auch jeder beliebige Datentyp sein)

```
class MeineKlasse{
```

```
    void machEtwas ( [...] ) {  
        [Anweisungen der Methode]  
    }  
}
```

Methodenbezeichner (**KLEINER** Anfangsbuchstabe)

Methode **machEtwas ()**

Methoden können generell zwei unterschiedlichen Zielen dienen:

- Änderung des **Objektzustands**
- Ausführung einer bestimmten **Aufgabe**

Werte- und Referenzparameter

Durch die **Parameterliste (Signatur)** einer Methode können zwei Arten von Parametern übergeben werden:

1. Werteparameter

Bei Variablen eines **elementareren Datentyps und Strings** handelt es sich um einen Werteparameter, da der Wert der Variablen **kopiert** wird und als **lokale Variable** in die Methode eingeht.

2. Referenzparameter

Bei Variablen eines **komplexen Datentyps** (also eines Objekts) handelt es sich um einen Referenzparameter, da in der Methode eine neue Referenz auf das **übergebene Objekt** erzeugt wird.

Lokale Variablen in Methoden

Methoden können auf alle Attribute des Objekts zugreifen und erhalten Daten über die Parameterliste. Darüber hinaus können Sie auch noch **explizit deklarierte lokale Variablen** besitzen:

```
class MeineKlasse{  
    int meinAttribut = 10;  
  
    void machEtwas(int einParameter) {  
        int lokaleVariable = meinAttribut + einParameter;  
    }  
}
```

Attribut der Klasse MeineKlasse

Implizite lokale Variable der Methode machEtwas()

Explizite lokale Variable der Methode machEtwas()

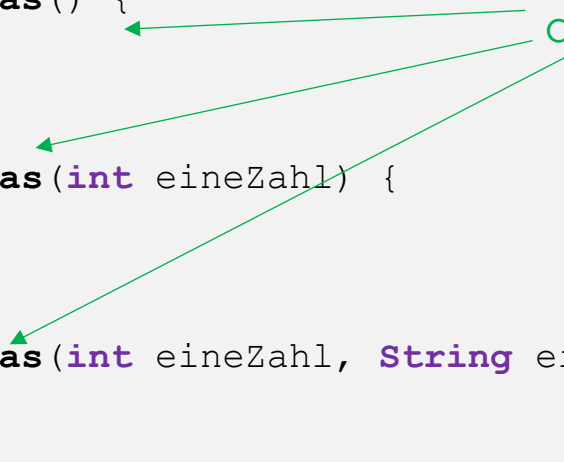
1. Lokale Variablen sind nur innerhalb ihrer Methode zugreifbar!
2. Werteparameter (Parameter elementarer Datentypen) sind ebenfalls lokale Variablen!

Überladung von Methoden

Eine Methode, die mehrfach, also mit gleichem Bezeichner aber mit **unterschiedlichen Parameterlisten** (Signaturen) deklariert wird, wird eine **überladene Methode** genannt:

```
class MeineKlasse{  
    void machEtwas() {  
        [...]  
    }  
  
    void machEtwas(int eineZahl) {  
        [...]  
    }  
  
    void machEtwas(int eineZahl, String einString) {  
        [...]  
    }  
}
```

Drei unterschiedliche Ausprägungen
der (überladenen) Methode machEtwas()



Aufgabe 6.3



Ergänzen Sie Ihr Programm **Pferd** um die Methode `traben()` ;
Diese Methode soll die **strecke** die das Pferd zurück gelegt hat um
den Wert der **geschwindigkeit** erhöhen (optional Geschw. zufällig
variieren).

Geben Sie beim Traben den Namen des Pferdes in der Konsole aus
und wie viele Meter es zurück gelegt hat.

In der Main Methode können wir mit
zwei Pferden ein Rennen durchführen
und sehen wer gewonnen hat.

Beispiel:

```
Jolly trabt 0,9 Meter
```

```
Wendy trabt 1,3 Meter
```

```
...
```

```
Jolly trabt 1,0 Meter
```

```
Wendy trabt 1,7 Meter
```

```
Jolly Strecke 8.4 Meter
```

```
Wendy Strecke 10.7 Meter
```

Rückgabewert von Methoden

Methoden können zur Erledigung ihrer Aufgabe auch einen **Wert zurückgeben**. Der Rückgabewert kann einen elementaren Datentyp oder ein komplexer Datentyp (also ein Objekt) sein.

Dazu wird der Rückgabe-Datentyp dem Methodenbezeichner im Kopf der Methode **vorangestellt**. Der Wert selber wird innerhalb der Methode mit dem Schlüsselwort **return** zurückgegeben:

Vorangestellter Rückgabe-Datentyp

```
int multipliziere(int faktorA, int faktorB) {  
  
    int produkt = faktorA * faktorB;  
  
    return produkt; // alternativ: return faktorA * faktorB  
}
```

Rückgabe-
Schlüsselwort (führt
zugleich zur
Beendigung der
Methode)

Aufgabe 6.4



- a. Ergänzen Sie Ihr Programm **Pferd** um die **public**-Methode `toString()` ;

Diese soll die Daten des Pferds als String zurück liefern (Rückgabewert: **String**).

Beispiel:

```
Pferd jolly = new Pferd("Jolly", 1);  
System.out.println(jolly.toString());
```

Mögliche Ausgabe:

Jolly – Gesch: 1.0 Strecke: 10

- b. Pferde sollen beim *Traben* jetzt zufällig hungrig werden und dann langsamer werden.

Statische Attribute

Statische Attribute (auch "Klassenvariablen" genannt) sind Attribute, die **direkt zur Klasse gehören** und **losgelöst** von Instanzen existieren. Sie werden durch Voranstellen des Schlüsselworts **static** deklariert:

```
class MeineKlasse{  
    static int statischeZahl = 10;  
}
```

Statische Variable der Klasse **MeineKlasse**

1. Der Zugriff auf statische Attribute erfolgt über den **Klassenbezeichner**.
2. Auf statische Attribute kann zugegriffen werden, auch wenn von der zugehörigen Klasse **keine Instanzen** existieren.
3. Statische Attribute werden bei Instanziierung nicht kopiert. Sie tragen somit **Instanz übergreifende Informationen**.

Statische Methoden

Statische Methoden werden – wie statische Attribute – ohne Existenz von Instanzen der Klasse verwendet (aufgerufen) und werden durch Voranstellen des Schlüsselworts **static** definiert:

```
class MeineKlasse{  
    static void machEtwasKlassenSpezifisches() {  
        [...]  
    }  
}
```

Statische Methode der Klasse **MeineKlasse**

1. Der Aufruf statischer Methoden erfolgt über den **Klassenbezeichner**.
2. Statische Methoden können **nur auf statische Attribute** oder andere **statische Methoden** der Klasse zugreifen.
3. Statische Methoden werden oft für generelle Aufgaben verwendet.

Aufgabe 6.5



- a. Ergänzen Sie das **statische** Attribut ***allePferde*** vom Typ **`List<Pferd>`** in der Klasse **Pferd**.
- b. Im Konstruktor von **Pferd** soll das vom Konstruktor erzeugte Objekt (**Pferd**) der Liste **allePferde** hinzugefügt werden.
- c. Außerdem soll eine **statische** Methode (ohne Rückgabe oder Übergabewerte) mit Namen **`resetPony()`** angelegt werden, welche die **strecke** aller **Pferde** in ***allePferde*** auf 0 zurück setzt.

(optional) Erstelle eine statische Methode **`racePony(double distance)`**, welche alle **Pferde** so lange traben lässt, bis eines die übergebene Distanz zurückgelegt hat.



Zugriffsmodifizierer

Zugriffsmodifizierer regeln, **ob** und wenn ja **welche** Elemente (Attribute und Methoden) einer Klasse von anderen Klassen adressiert (benutzt und/oder aufgerufen) werden können.

Konkret:

1. Von welchen Klassen im selben oder in anderen Paketen gestattet eine bestimmte Klasse Zugriff?
→ **Zugriffsmodifizierer für Klassen**
2. Falls Zugriff besteht: Auf welche ihrer Elemente (Attribute und Methoden) gestattet diese Klasse Zugriff?
→ **Zugriffsmodifizierer für Klassenelemente**

Zugriffsmodifizierer für **Klassen**

public

Zugriff von Klassen in **allen Paketen**

keine Angabe (Standard)

Zugriff von Klassen innerhalb **desselben Paketen**

Zugriffsmodifizierer für Klassenelemente*

public

Zugriff von Klassen in **allen Paketen**

protected

Wie Standard plus Zugriff von **abgeleiteten Klassen** (auch) **in anderen Paketen**

keine Angabe (Standard)

Zugriff von Klassen innerhalb **desselben Paketes**

private

Zugriff nur von innerhalb **derselben Klasse**

Viele Dank für Ihre Aufmerksamkeit



Hausaufgabe



Schreiben Sie eine Klasse **Motorrad**, mit der Sie Motorräder mit und ohne explizite Eingabe von Startwerten instanziiieren können. Ihre Klasse sollte folgende Attribute besitzen:

Attribut	Mögliche Werte
typ	Sportler, Chopper, Cruiser, Enduro
farbe	schwarz, rot, blau, silber
hubraum	Ganzzahl zwischen 125 und 1300 (ccm)

Darüber hinaus sollte Ihre Klasse folgende Methoden haben:

Methode	Beschreibung	Rückgabewert
void schreibeInfos()	Tabellarische Ausgabe über Typ, Farbe und Hubraum	kein
double berechneSteuern()	Steuern pro Jahr (Regel: 1,84 € pro 25 ccm)	Steuern in €